

Gestion d'une file d'attente dans une agence bancaire

*Implémentation d'une structure de données linéaire (File / Queue) en Java, avec interface graphique
JavaFX*

Niveau	L2 / L3 — Informatique (adaptable DUT / BTS)
Langage	Java (JDK 17+)
Interface graphique	JavaFX (SDK 21 recommandé)
Structure étudiée	File d'attente (Queue) — principe FIFO (First In, First Out)
Durée indicative	2 séances de 3 heures (TP) + travail personnel
Mode de travail	Individuel ou binôme
Livrables	Code source (projet Maven/Gradle), rapport, capture(s) d'écran

1. Objectifs pédagogiques

À l'issue de ce TP, l'étudiant doit être capable de :

- Expliquer le fonctionnement et les propriétés d'une file d'attente (FIFO) et la distinguer d'une pile (LIFO) ;
- Implémenter une file générique en Java à partir d'une structure interne (tableau circulaire ou liste chaînée), sans utiliser directement `java.util.Queue` pour la logique métier ;
- Concevoir une architecture logicielle simple séparant la logique métier (modèle) de l'interface graphique (vue), selon un esprit MVC ;
- Développer une interface graphique interactive avec JavaFX permettant de manipuler la structure en temps réel ;
- Mettre en œuvre les opérations fondamentales : enfiler (enqueue), défiler (dequeue), consulter la tête de file (peek), tester la file vide/pleine, calculer la taille.

2. Prérequis

- Notions de programmation orientée objet en Java (classes, héritage, interfaces, génériques) ;
- Notions de base sur les structures de données linéaires (tableaux, listes chaînées) ;
- Bases de JavaFX : Stage, Scene, conteneurs (VBox, HBox, BorderPane), contrôles (Button, TextField, ListView), gestion d'événements.

3. Contexte et mise en situation

La Direction des Systèmes d'Information d'une agence bancaire souhaite informatiser la gestion du flux de clients au guichet. Actuellement, les clients patientent sans dispositif de gestion automatisée : aucun ordre de passage fiable n'est garanti, ce qui provoque des tensions et des pertes de temps.

Vous êtes chargé(e) de développer une application de bureau permettant de gérer la file d'attente des clients d'une agence, du moment où le client se présente (tickets) jusqu'à sa prise en charge par un guichetier. L'application doit respecter le principe FIFO : le premier client arrivé est le premier servi.

4. Rappels théoriques

4.1. Définition

Une file (queue) est une structure de données linéaire fonctionnant selon le principe FIFO (First In, First Out) : les éléments sont insérés à une extrémité (l'arrière, ou « rear ») et retirés à l'autre extrémité (l'avant, ou « front »).

4.2. Opérations fondamentales

Opération	Description	Complexité attendue
enfiler(x)	Ajoute l'élément x à l'arrière de la file	O(1)
defiler()	Retire et retourne l'élément à l'avant de la file	O(1)

Opération	Description	Complexité attendue
tete()	Retourne l'élément en tête sans le retirer	O(1)
estVide()	Indique si la file ne contient aucun élément	O(1)
taille()	Retourne le nombre d'éléments présents	O(1)

4.3. Choix d'implémentation possibles

- Tableau circulaire (buffer circulaire) : tableau de taille fixe ou redimensionnable, avec deux indices tete et queue évoluant modulo la capacité ;
- Liste chaînée simple : chaque élément (nœud) référence le suivant ; insertion en queue et suppression en tête en temps constant grâce à des pointeurs premier/dernier.

Le choix de l'implémentation interne est laissé à l'étudiant, mais doit être justifié dans le rapport (avantages/inconvénients en mémoire et en performance).

5. Spécifications fonctionnelles

L'application doit permettre les fonctionnalités suivantes :

1. Prise de ticket : un client arrive, saisit son nom et choisit un motif de visite (Dépôt, Retrait, Ouverture de compte, Virement, Autre). Un ticket lui est attribué automatiquement avec un numéro incrémental et une heure d'arrivée.
2. Appel du client suivant : le guichetier clique sur un bouton « Appeler client suivant » ; le client en tête de file est retiré et affiché comme « en cours de traitement ».
3. Visualisation de la file : la liste des clients en attente est affichée en temps réel, dans l'ordre FIFO, avec leur numéro de ticket, nom, motif et heure d'arrivée.
4. Annulation : un client peut quitter la file avant d'être appelé (retrait à une position quelconque de la file, pas uniquement en tête).
5. Statistiques simples : nombre de clients en attente, temps d'attente moyen estimé, nombre total de clients servis depuis le lancement de l'application.
6. Gestion de capacité (si tableau circulaire) : si la file est pleine, l'application doit avertir l'utilisateur plutôt que de planter.

Fonctionnalité bonus (facultative, voir section 8) :

- Gestion de plusieurs guichets (plusieurs files ou une file unique répartie sur guichets disponibles) ;
- Priorité VIP : une file prioritaire pour certains clients (personnes âgées, clients premium), traitée avant la file standard.

6. Spécifications techniques

6.1. Architecture proposée

Le projet doit être organisé selon une séparation claire entre le modèle (structure de données et logique métier) et la vue (interface JavaFX), par exemple :

```
src/
  main/
    java/
      modele/
        Client.java
        IFile.java           (interface générique de la structure)
        FileCirculaire.java  (implémentation par tableau circulaire)
        FileChaine.java     (implémentation par liste chaînée)
        GestionnaireFileAttente.java (logique métier : tickets, stats)
      dao/
        ConnexionBD.java
        ClientServiDAO.java
        ClientServiDAOImpl.java
      vue/
        MainApp.java         (point d'entrée JavaFX, extends Application)
        VueGuichet.java      (construction de l'interface)
        ControleurGuichet.java
      resources/
        styles.css
        config.properties    (URL JDBC, utilisateur, mot de passe)
      script/
        schema.sql           (script de création de la base de données)
```

6.2. Squelette de l'interface générique

La structure de file doit être manipulée via une interface générique, indépendante de son implémentation interne :

```
public interface IFile<T> {
    void enfiler(T element);
    T defiler();
    T tete();
    boolean estVide();
    boolean estPleine(); // pertinent surtout pour le tableau circulaire
    int taille();
}
```

6.3. Classe métier Client

```
public class Client {
    private final int numeroTicket;
    private final String nom;
    private final String motif;
    private final LocalDateTime heureArrivee;
    // constructeur, getters, toString()...
}
```

6.4. Interface graphique (JavaFX)

L'interface doit comporter au minimum :

- Une zone de saisie (nom du client, motif) et un bouton « Prendre un ticket » ;
- Un bouton « Appeler client suivant », désactivé automatiquement lorsque la file est vide ;
- Un composant ListView<Client> (ou TableView) affichant la file en cours, mis à jour à chaque opération ;
- Une zone affichant le client actuellement en cours de traitement au guichet ;
- Un panneau de statistiques (nombre en attente, clients servis, temps d'attente moyen) ;
- Une mise en forme soignée (utilisation d'un fichier styles.css, mise en page avec BorderPane/VBox/HBox).

6.5. Contraintes techniques

- La logique de la file (enfiler/défiler) doit être codée « à la main » dans FileCirculaire ou FileChaine : l'usage direct de java.util.LinkedList ou java.util.ArrayDeque pour stocker les clients n'est pas autorisé dans la couche modèle (il peut en revanche être utilisé pour peupler une ListView côté vue si nécessaire) ;
- Le code doit être commenté (Javadoc sur les classes et méthodes publiques) ;
- Gestion des cas limites : défiler() ou tete() sur une file vide doit lever une exception explicite (ex. FileVideException) plutôt que provoquer un comportement indéfini ;
- Respect des conventions de nommage Java (camelCase, PascalCase pour les classes).

6.6. Persistance des données (base de données)

La persistance n'est plus réalisée en fichier plat : elle doit s'appuyer sur une base de données relationnelle, accédée en Java via JDBC. Elle a deux objectifs : conserver l'historique des clients servis (traçabilité) et permettre de recharger les statistiques au redémarrage de l'application (le compteur de tickets et les clients servis ne doivent pas repartir de zéro à chaque lancement).

SGBD imposé : MySQL / MariaDB (ou PostgreSQL, au choix de l'enseignant). Un script SQL de création du schéma doit être fourni avec le projet.

Schéma minimal proposé :

```
CREATE TABLE client_servi (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  numero_ticket INT NOT NULL,
  nom         VARCHAR(100) NOT NULL,
  motif       VARCHAR(50)  NOT NULL,
  priorite    BOOLEAN      NOT NULL DEFAULT FALSE,
  heure_arrivee DATETIME    NOT NULL,
  heure_prise_en_charge DATETIME NOT NULL,
  guichet     VARCHAR(30)
);
```

Organisation du code : la couche d'accès aux données doit être isolée dans un package dédié (dao), selon le patron DAO (Data Access Object), pour ne pas mélanger SQL et logique métier :

```
modele/
  dao/
    ConnexionBD.java      (ouverture/fermeture de la connexion JDBC)
    ClientServiDAO.java   (interface : enregistrer, listerHistorique, ...)
    ClientServiDAOImpl.java (implémentation JDBC, requêtes préparées)
```

- La connexion doit utiliser des requêtes préparées (PreparedStatement) — jamais de concaténation de chaînes dans le SQL (risque d'injection SQL) ;
- Chaque client appelé au guichet (dequeue) doit être inséré dans client_servi avec son heure de prise en charge ;
- Au démarrage de l'application, le nombre total de clients servis et le prochain numéro de ticket disponible doivent être recalculés à partir de la base de données ;
- Les paramètres de connexion (URL JDBC, utilisateur, mot de passe) doivent être externalisés dans un fichier de configuration (ex. config.properties), jamais codés en dur dans les classes.

7. Travail demandé

Partie A — Couche métier (structure de données)

7. Définir l'interface IFile<T> et choisir une implémentation (tableau circulaire ou liste chaînée), en justifiant ce choix.
8. Implémenter la classe Client et la classe de file choisie, avec gestion des exceptions (file vide, file pleine le cas échéant).
9. Écrire une classe de test (main ou tests unitaires JUnit) qui enfile 5 clients, en défile 2, puis affiche l'état de la file, afin de valider le bon fonctionnement avant de passer à l'interface graphique.

Partie B — Interface graphique JavaFX

10. Mettre en place le squelette de l'application JavaFX (classe MainApp, fenêtre principale).
11. Implémenter le formulaire de prise de ticket et la mise à jour dynamique de l'affichage de la file.
12. Implémenter le bouton d'appel du client suivant et l'affichage du client en cours de traitement.
13. Implémenter le panneau de statistiques et la fonctionnalité d'annulation d'un ticket.
14. Soigner la présentation (CSS, icônes ou couleurs pour différencier les motifs de visite, etc.).

Partie C — Bonus (facultatif, points supplémentaires)

15. Ajouter une file prioritaire VIP traitée avant la file standard, ou la gestion de plusieurs guichets en parallèle.
16. Ajouter la persistance en base de données décrite en section 6.6 (historique des clients servis, statistiques rechargeables au démarrage).

8. Livrables attendus

- Le code source complet du projet (archive .zip ou dépôt Git), structuré comme indiqué en 6.1, incluant le script SQL de création du schéma (schema.sql) ;
- Un court rapport (2 à 4 pages) présentant : le choix d'implémentation de la file et sa justification, un schéma UML simplifié des classes du modèle, le schéma de la base de données, une ou deux captures d'écran de l'application en fonctionnement, les difficultés rencontrées ;
- Le tout déposé selon les modalités communiquées par l'enseignant (plateforme de dépôt / clé USB / email), avant la date limite indiquée en séance.

9. Barème indicatif

Critère	Points	
Correction et robustesse de l'implémentation de la file (Partie A)	6	
Fonctionnalités de l'interface JavaFX (Partie B)	7	
Qualité du code (structure, nommage, commentaires)	3	
Rapport (justification, UML, captures d'écran)	3	
Bonus (Partie C)	+1	
Total	20	

Bon travail !